Wearable “SupaClock”

Reporte de Avance 25 % | Validación de Subsistemas

Grupo 10

Ingeniería Eléctrica

30 de abril de 2026

Contexto, Requisitos y Solución

Problemática:

- Alta tasa de sedentarismo y sobrepeso (42 % de la población).
- Necesidad de fomentar hábitos saludables mediante monitoreo preventivo y accesible.

Requisitos de Diseño:

- **Factor de forma**: *Smartwatch* compacto ($<80\times50\times30$ mm) y autónomo (>12 h).
- **Biometría 5-en-1**: SpO_2, Ritmo Cardíaco, Temperatura, Podómetro, ECG monocanal.
- **Procesamiento Edge**: Inteligencia Artificial local para clasificar actividad.

Solución Propuesta: *SupaClock*

- Wearable de monitoreo continuo con enfoque clínico ("Cero distracciones").
- **Hardware**: ESP32 (C3/S3), 6 sensores I^2C/ADC y pantalla LCD a color (SPI).
- **Ecosistema**: Telemetría BLE en vivo hacia App y procesamiento (Pan-Tompkins) en la nube.

Figura 1: *Renderizado 3D del concepto SupaClock.*

Agenda de la presentación

1. **Hardware e integración**
2. **Firmware y algoritmos**
3. **Comunicación, métricas y futuro**

Hardware e Integración

Ecosistema de Hardware: 6 sensores integrados

Componente	Bus
BMI160 (IMU 6-DOF)	I^2C 0x68
MAX30102 (PPG/HR/SpO_2)	I^2C 0x57
MAX30205 (Temp corporal)	I^2C 0x48
MAX17048 (Fuel gauge)	I^2C 0x36
AD8232 (ECG)	ADC1 + DMA
ST7789 (Display 240×280)	SPI 80 MHz

- Bus I^2C compartido a **400 kHz**
- Mutex global protege accesos concurrentes
- ECG por DMA continuo (no compite por I^2C)

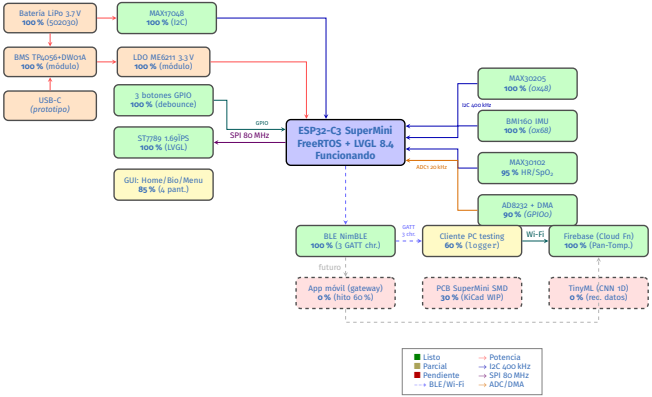

Figura 2: *Diagrama de bloques de hardware.*

Subsistema de Energía y PCB Prototipo

- **BMS**: TP4056 (carga LiPo) + DW01A (protección sobre/sub-tensión)
- **Fuel gauge**: MAX17048 reporta voltaje y SoC por I^2C
- **Regulador (C3 SuperMini)**: LDO ME6211 **insuficiente** para picos de radio + sensores

Problema detectado

Reset por brownout al activar BLE concurrente con bursts del display. Pico medido $\sim$**110 mA** en batería (probablemente subestimado por la respuesta del amperímetro).

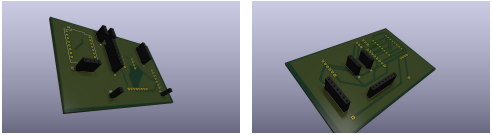

Figura 3: *PCB through-hole con C3 SuperMini en montaje; en paralelo se encargó la revisión siguiente con ESP32-S3 (slide 12).*

Firmware y Algoritmos

Arquitectura de Software: 6 tareas FreeRTOS + Modos de Energía

Tarea	Prio	Cadencia
`ecg_task`	7	DMA continuo
`imu_task`	6	50/25/12.5 Hz*
`hrm_task`	5	SM (modo)
`gui_task`	5	30 FPS
`ble_tx_task`	4	flush periódico
`system_task`	3	30 s / 5 min

**según modo de energía activo.*

Sincronización: `xSensorDataMutex` y mutex global del bus I^2C.

	SPORT	NORMAL	SAVER
HR (MAX30102)	continuo	cada 10 min	cada 30 min
SpO_2	cada 5 min	cada 30 min	manual
IMU	50 Hz	25 Hz	12.5 Hz
Temp	30 s	5 min	15 min
Batería	30 s	30 s	30 s
BLE flush	1 s	10 s	60 s
Auto-off	30 s	15 s	8 s

- Selección manual desde menú; persistencia en NVS
- En NORMAL/SAVER, MAX30102 entra a SHDN ($\sim$0.7 μA)
- Toggle “Tx IMU” apaga radio sin perder podometría
- Apagado total vía *deep sleep* con wake por botón

Flujo de Tareas y Sensores concurrentes en FreeRTOS

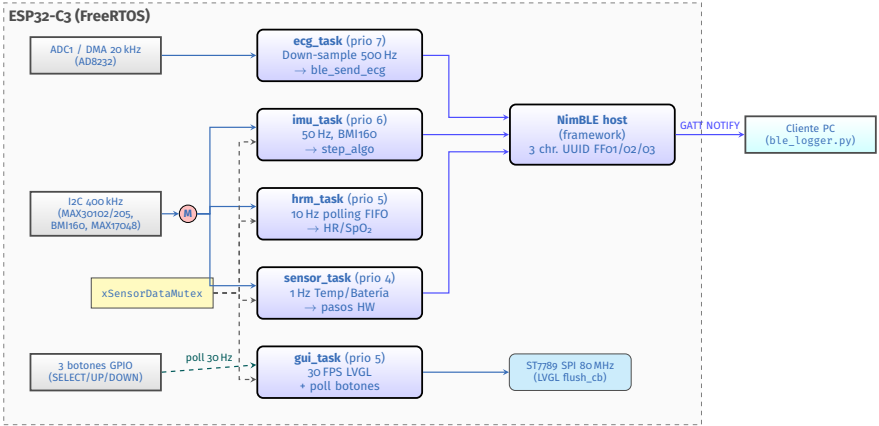

Figura 4: *Flujo de tareas y sensores en FreeRTOS.*

Pipeline HR/SpO_2: bandpass IIR + mediana

Captura: Red + IR a 25 Hz (SMP_AVE=4).

Pipeline:

1. Tracker DC (EMA α=0.05)
2. Bandpass IIR Butterworth **0.5–4 Hz** (HPF+LPF orden 2)
3. Detector picos umbral adaptativo
4. Mediana 8 RR + rechazo ΔRR>25 %
5. SpO_2: mediana 10 valores R
6. Motion gating con jerk del IMU

SPOT dedicado: 12–30 s con early-exit por calidad (σ/μ RR<8 %, R<5 %).

Validación 30 s reposo:
HR **81 BPM** (33 picos) | SpO_2 **98.7 %**

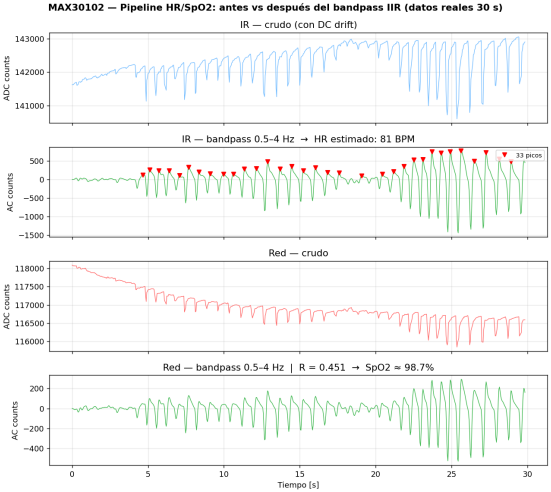

Figura 5: *Algoritmo PPG para HR y SpO_2.*

ECG: Adquisición y Pan-Tompkins

Adquisición:

- AD8232 + ADC continuo con DMA a **20 kHz**
- Diezmado por factor $\times 40 \rightarrow$ stream BLE a **500 Hz**
- Característica BLE 0xFF03 dedicada

Pan-Tompkins (offline en host):

1. Bandpass 5–15 Hz (banda QRS)
2. Derivada (resalta pendientes)
3. Cuadratura
4. Integración en ventana móvil
5. Detección de picos R con umbral adaptativo

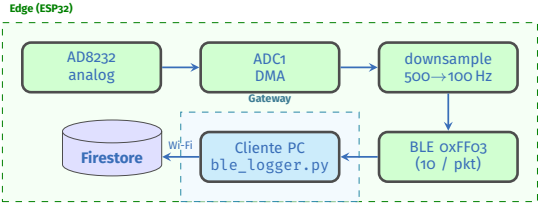

Figura 6: *Adquisición de ECG (Edge).*

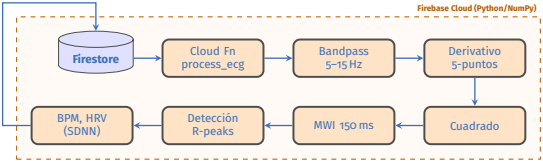

Figura 7: *Procesamiento Pan-Tompkins (Cloud).*

Podómetro: del dominio del tiempo al de la frecuencia

Implementación actual (dominio tiempo):

1. Magnitud 3D del acelerador
2. Filtro media móvil (10 muestras)
3. Detector de picos con umbral adaptativo (re-evaluación cada 5 s)

Limitación detectada: agitar la mano emula picos → falsos positivos. **Solución planificada** (dominio frecuencia):

- FFT sobre ventana de 3 s
- Validar cadencia rítmica (1–3 Hz)
- Requiere DSP vectorial → ESP32-S3

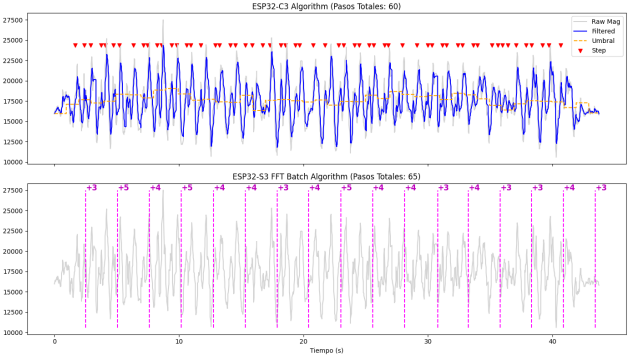

Figura 8: *Algoritmo en el dominio del tiempo.*

Validación por subsistemas: 9 environments PlatformIO

Cada sensor / módulo cuenta con su propio firmware de prueba aislado, más un firmware integrador (`test_general`).

Env	Subsistema
`test_temp`	MAX30205
`test_imu`	BMI160 + step_algo
`test_spo2`	MAX30102 raw + BLE
`test_ecg`	AD8232 ADC/DMA
`test_display`	ST7789 + LVGL
`test_ble`	NimBLE GATT
`test_fuel_gauge`	MAX17048
`test_gui`	Navegación GUI
`test_general`	**Integrador**

- Cada `test_*` se compila y flashea independiente
- Permite aislar fallas de driver vs integración
- `test_general` ejecuta los 6 sensores + GUI + BLE simultáneamente → valida concurrencia, mutex, modos de energía
- Demo en vivo se hace con `test_general`

Comunicación, Métricas y Futuro

Stack BLE y herramienta de validación

4 características BLE (GATT custom 0xFF00):

- `0xFF01` IMU 6-DOF (12 B/notify, 50 Hz)
- `0xFF02` Telemetría agregada **TLV** (HR, SpO_2, temp, bat, modos)
- `0xFF03` ECG streaming (chunks 500 Hz)
- `0xFF04` Comandos RX (host → reloj)

El protocolo TLV permite agrupar muestras heterogéneas con timestamp, reduciendo el overhead BLE en SAVER.

screenshot `supaclock_monitor.py`
(PyQt6 + bleak + pyqtgraph + OpenGL)
(pendiente)

Figura 9: ***supaclock_monitor.py*** *muestra biometría en vivo y graba CSV para entrenar clasificador CNN 1D.*

Optimización energética: antes vs después

Estados del SoC (Power Manager)

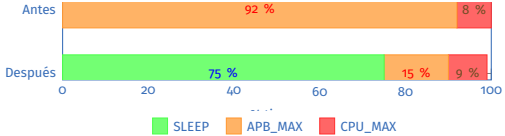

Figura 10: *Estados de energía del SoC.*

Consumo en batería (Tektronix TX3)

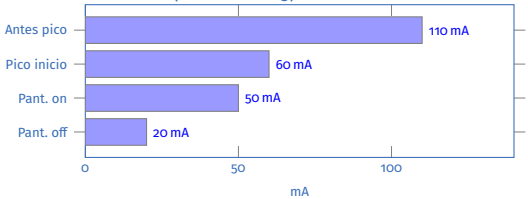

Figura 11: *Picos de consumo en batería.*

Mitigaciones aplicadas

- Driver ST7789 reescrito: `gui_task` 47 % → **1 %**
- Handle ADC del ECG asignado dinámicamente (libera lock APB_MAX)
- Advertising BLE a 1000 ms y Modem Sleep (light sleep con conexión)
- Toggle "Tx IMU" apaga radio manteniendo podometría local

CPU por tarea (después)

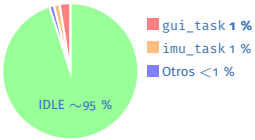

Figura 12: *CPU ya no satura; el cuello de botella pasa a ser memoria.*

Problemáticas y Limitaciones del Prototipo Actual

Durante la integración de biosensores con el **ESP32-C3**, se identificaron barreras técnicas críticas:

- **Inestabilidad de Potencia (Brownouts)**: El LDO actual ($\sim$300 mA) no soporta picos concurrentes de transmisión BLE y la retroiluminación LCD, causando reinicios súbitos.
- **Techo de Memoria (SRAM)**: Tras inicializar Bluetooth y FreeRTOS, restan solo $\sim$99 KB de RAM. Esto asfixia el despliegue del modelo de IA (*TinyML*) para clasificar actividad.
- **Déficit de Hardware Vectorial**: El procesador *single-core* carece de aceleración SIMD nativa, provocando que la inferencia de *Machine Learning* demore hasta 10 veces más ($\sim$200 ms).
- **Baja Precisión en Podometría**: El algoritmo actual de detección de pasos por umbral simple (magnitud IMU) es impreciso y genera falsos positivos con movimientos cotidianos.

Impacto en el Proyecto

El firmware y la concurrencia son estables, pero el **hardware del ESP32-C3** es un cuello de botella físico para lograr la autonomía de IA local.

Migración al Seeed XIAO ESP32-S3 (encargada)

Tras eliminar a la CPU como bottleneck, hay 5 razones técnicas concretas para migrar la siguiente revisión de PCB al ESP32-S3.

1. **Brownout resuelto**: Buck SGM6029 integrado (∼800 mA) reemplaza al LDO ME6211 (∼300 mA).
2. **Memoria para ML**: 512 KB SRAM (vs 320 en C3) + PSRAM externa hasta 8 MB.
3. **ESP-NN con SIMD**: inferencia CNN 1D int8 **5–20 ms** (S3) vs 50–200 ms (C3).
4. **Dual-core 240 MHz**: separa ML del firmware sin contención con `ecg_task` / `gui_task`.
5. **BMS + fuel gauge integrados** en placa → menos componentes en PCB final.

Memoria disponible

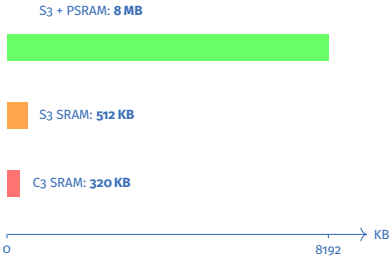

Figura 13: *Escala lineal: la PSRAM externa del S3 abre ∼25× más memoria que el C3.*

Heap actual vs CNN

Heap libre en C3: ∼99 KB. Modelo CNN 1D + scratch: 20–100 KB. Margen ajustado → S3 abre la puerta a modelos grandes.

Próximos pasos y modelo de Machine Learning

Clasificador de actividad — CNN 1D

- **Entrada**: ventana 3 s del IMU $\rightarrow$ 6 canales $\times$ 150 muestras
- **Arquitectura**: 2–3 capas Conv1D + GAP + Dense
- **Salida**: 4–5 clases (reposo / caminar / correr / agitar mano / dormir)
- Entrenamiento offline con CSV recolectados vía `supaclock_monitor.py`
- Despliegue en S3 vía ESP-NN cuantizada int8

Hito 50 %

1. Validación de la S3 con todos los drivers actuales
2. Migración del podómetro a FFT (ESP-DSP)
3. Recolección dataset de actividades reales
4. Entrenamiento + cuantización CNN 1D
5. PCB SMD con S3 + sensores integrados

Demostración en vivo

5 min

Navegación UI | HR continuo | SPOT dedicado | cambio de modo | ECG streaming
Telemetría BLE en `supaclock_monitor.py`